Generative Score-Based Models: Reverse Engineering Noise

Vinay Joshy

March 31, 2025

Image Generation

Generated using ChatGPT

Probability distribution of cats

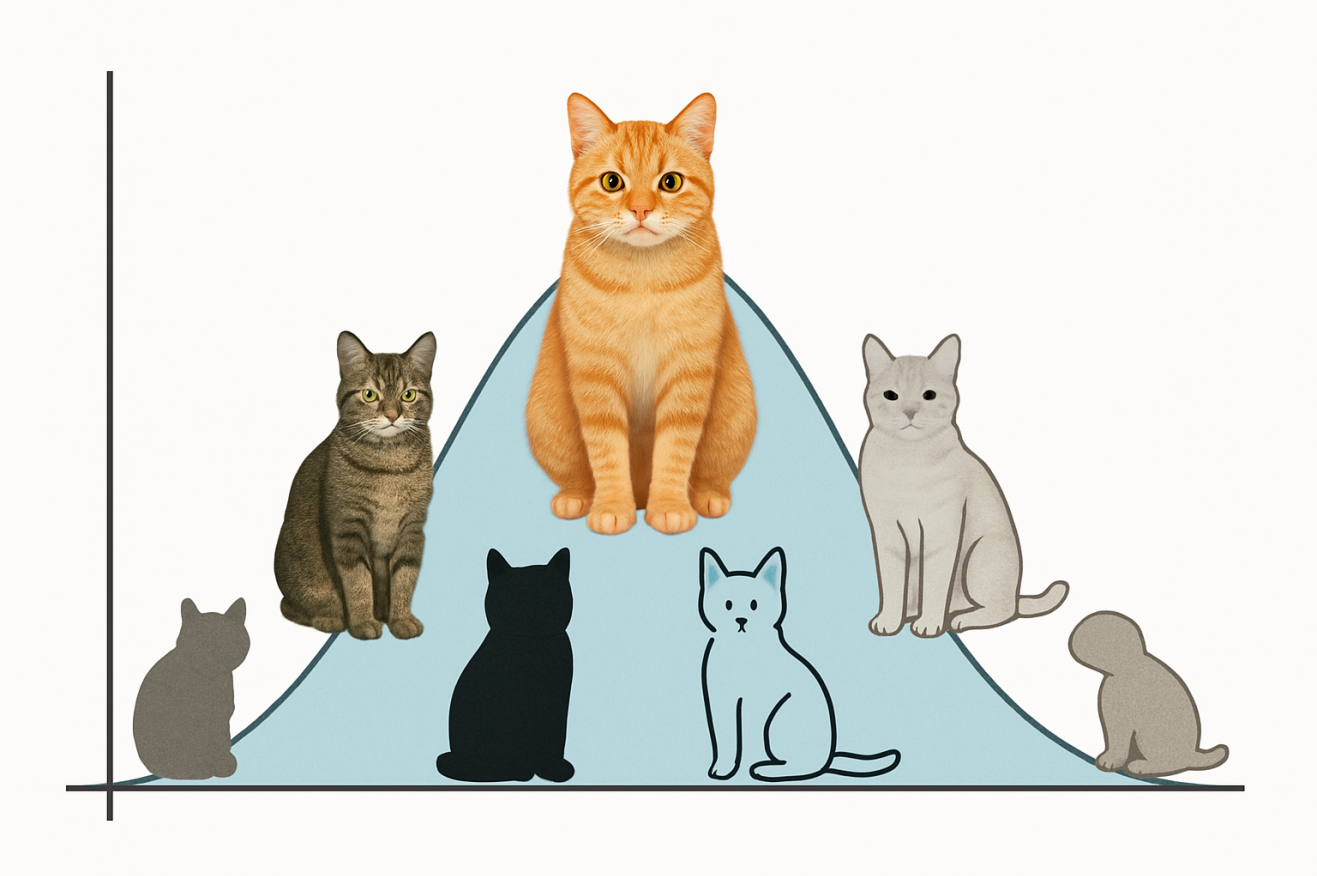


Figure: Generated using ChatGPT

Something from nothing

- $\mathbf{x}_1, \ldots, \mathbf{x}_N \sim p_d(\mathbf{x})$

Something from nothing

- $\mathbf{x}_1, \ldots, \mathbf{x}_N \sim p_d(\mathbf{x})$
- $p_d(\mathbf{x})$ is the data distribution

Something from nothing

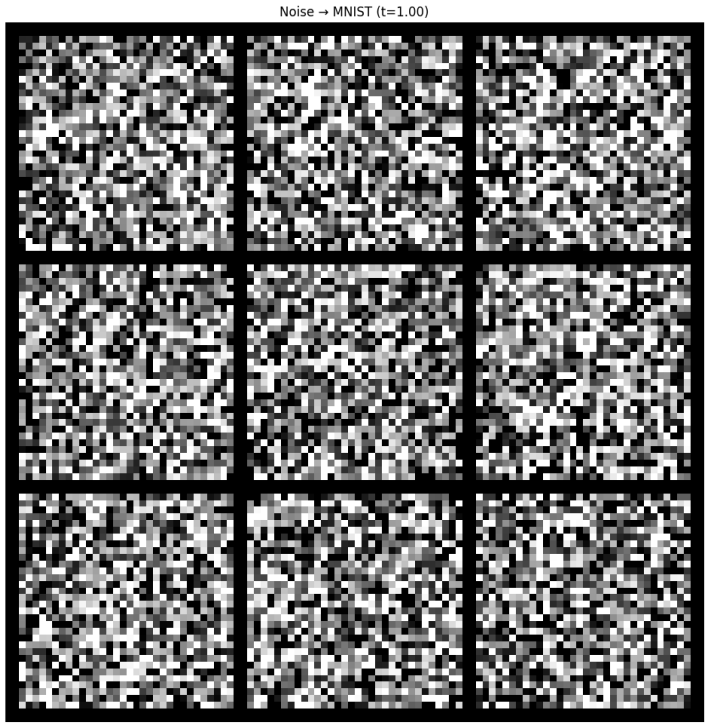
- $\mathbf{x}_1, \ldots, \mathbf{x}_N \sim p_d(\mathbf{x})$
- $p_d(\mathbf{x})$ is the data distribution
- $\mathbf{z}_1, \ldots, \mathbf{z}_N \sim p_{\text{init}}(\mathbf{z})$, e.g., $p_{\text{init}}(\mathbf{z}) = \mathcal{N}(\mathbf{0}, \mathbf{I}_D)$

Something from nothing

- $\mathbf{x}_1, \ldots, \mathbf{x}_N \sim p_d(\mathbf{x})$
- $p_d(\mathbf{x})$ is the data distribution
- $\mathbf{z}_1, \ldots, \mathbf{z}_N \sim p_{\text{init}}(\mathbf{z})$, e.g., $p_{\text{init}}(\mathbf{z}) = \mathcal{N}(\mathbf{0}, \mathbf{I}_D)$
- GOAL: Transform samples from $p_{\text{init}}(\mathbf{z})$ into samples from $p_d(\mathbf{x})$

Something from nothing

- $\mathbf{x}_1, \ldots, \mathbf{x}_N \sim p_d(\mathbf{x})$
- $p_d(\mathbf{x})$ is the data distribution
- $\mathbf{z}_1, \ldots, \mathbf{z}_N \sim p_{\text{init}}(\mathbf{z})$, e.g., $p_{\text{init}}(\mathbf{z}) = \mathcal{N}(\mathbf{0}, \mathbf{I}_D)$
- GOAL: Transform samples from $p_{\text{init}}(\mathbf{z})$ into samples from $p_d(\mathbf{x})$



Learning Unnormalized Models

- How do we find this transformation from $p_{\text{init}}(\mathbf{z})$ to $p_d(\mathbf{x})$?

Learning Unnormalized Models

- How do we find this transformation from $p_{\text{init}}(\mathbf{z})$ to $p_d(\mathbf{x})$?
- **Problem setup:** Given i.i.d. samples $\{\mathbf{x}_1, \mathbf{x}_2, \cdots, \mathbf{x}_N\} \subset \mathbb{R}^D$ from data distribution $p_d(\mathbf{x})$

Learning Unnormalized Models

- How do we find this transformation from $p_{\text{init}}(\mathbf{z})$ to $p_d(\mathbf{x})$?
- **Problem setup:** Given i.i.d. samples $\{\mathbf{x}_1, \mathbf{x}_2, \cdots, \mathbf{x}_N\} \subset \mathbb{R}^D$ from data distribution $p_d(\mathbf{x})$
- Model an approximation of the data distribution, p_m.

Learning Unnormalized Models

- How do we find this transformation from $p_{\text{init}}(\mathbf{z})$ to $p_d(\mathbf{x})$?
- **Problem setup:** Given i.i.d. samples $\{\mathbf{x}_1, \mathbf{x}_2, \cdots, \mathbf{x}_N\} \subset \mathbb{R}^D$ from data distribution $p_d(\mathbf{x})$
- Model an approximation of the data distribution, p_m.
- This normalized density is then given by:

$$p_m(\mathbf{x}; \theta) = \frac{\tilde{p}_m(\mathbf{x}; \theta)}{Z_\theta}, \quad Z_\theta = \int \tilde{p}_m(\mathbf{x}; \theta) d\mathbf{x}$$

Learning Unnormalized Models

- How do we find this transformation from $p_{\text{init}}(\mathbf{z})$ to $p_d(\mathbf{x})$?
- **Problem setup:** Given i.i.d. samples $\{\mathbf{x}_1, \mathbf{x}_2, \cdots, \mathbf{x}_N\} \subset \mathbb{R}^D$ from data distribution $p_d(\mathbf{x})$
- Model an approximation of the data distribution, p_m.
- This normalized density is then given by:

$$p_m(\mathbf{x}; \theta) = \frac{\tilde{p}_m(\mathbf{x}; \theta)}{Z_\theta}, \quad Z_\theta = \int \tilde{p}_m(\mathbf{x}; \theta) d\mathbf{x}$$

- The partition function Z_θ makes it difficult to learn normalized models with maximum likelihood estimation.

Score Matching

- Hyvärinen, 2005 showed that score-matching minimizes the Fisher Divergence between $p_m(., \theta)$ and p_d, defined as

Score Matching

- Hyvärinen, 2005 showed that score-matching minimizes the Fisher Divergence between $p_m(., \theta)$ and p_d, defined as
-

$$L(\theta) \triangleq \frac{1}{2}\mathbb{E}[\| \mathbf{s}_m(x;\theta) - \mathbf{s}_d(x) \|_2^2]$$

Score Matching

- Hyvärinen, 2005 showed that score-matching minimizes the Fisher Divergence between $p_m(., \theta)$ and p_d, defined as
- $$L(\theta) \triangleq \frac{1}{2}\mathbb{E}[\| \mathbf{s}_m(x;\theta) - \mathbf{s}_d(x) \|_2^2]$$
- The Fisher divergence does not depend on the intractable partition function, Z_θ.

Score Matching

- Hyvärinen, 2005 showed that score-matching minimizes the Fisher Divergence between $p_m(.,\theta)$ and p_d, defined as
- $$L(\theta) \triangleq \frac{1}{2}\mathbb{E}[\| \mathbf{s}_m(x;\theta) - \mathbf{s}_d(x) \|_2^2]$$
- The Fisher divergence does not depend on the intractable partition function, Z_θ.
- The score function, $\mathbf{s}_m(x;\theta) = \nabla_x \log p_m(x;\theta)$

Score Matching

- Hyvärinen, 2005 showed that score-matching minimizes the Fisher Divergence between $p_m(.,\theta)$ and p_d, defined as
- $$L(\theta) \triangleq \frac{1}{2}\mathbb{E}[\| \mathbf{s}_m(x;\theta) - \mathbf{s}_d(x) \|_2^2]$$
- The Fisher divergence does not depend on the intractable partition function, Z_θ.
- The score function, $\mathbf{s}_m(x;\theta) = \nabla_x \log p_m(x;\theta)$
- We can learn the $p_m(.,\theta)$ by way of unnormalized density $\tilde{p}_m(\mathbf{x};\theta)$, where $\theta \in \Theta$

Score Matching

- Hyvärinen, 2005 showed that score-matching minimizes the Fisher Divergence between $p_m(.,\theta)$ and p_d, defined as
- $$L(\theta) \triangleq \frac{1}{2}\mathbb{E}[\| \mathbf{s}_m(x;\theta) - \mathbf{s}_d(x) \|_2^2]$$
- The Fisher divergence does not depend on the intractable partition function, Z_θ.
- The score function, $\mathbf{s}_m(x;\theta) = \nabla_x \log p_m(x;\theta)$
- We can learn the $p_m(.,\theta)$ by way of unnormalized density $\tilde{p}_m(\mathbf{x};\theta)$, where $\theta \in \Theta$
- **New Goal**: Estimate score function!

Noisy Business

- Direct score estimation on raw data is challenging

Noisy Business

- Direct score estimation on raw data is challenging
- Instead, we'll inject noise, σ_i, into our data at increasing levels $i = 1, ..., L$

Noisy Business

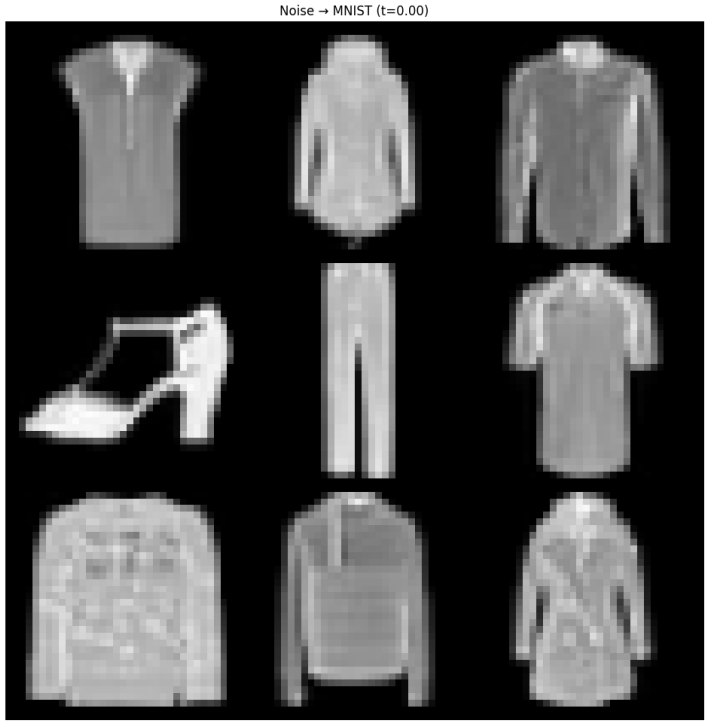
- Direct score estimation on raw data is challenging
- Instead, we'll inject noise, σ_i, into our data at increasing levels $i = 1, ..., L$
- Easier to learn the score of perturbed versions of the data at each level

Noisy Business

- Direct score estimation on raw data is challenging
- Instead, we'll inject noise, σ_i, into our data at increasing levels $i = 1, ..., L$
- Easier to learn the score of perturbed versions of the data at each level
- This procedure becomes a continuous-time stochastic process.

Noisy Business

- Direct score estimation on raw data is challenging
- Instead, we'll inject noise, σ_i, into our data at increasing levels $i = 1, ..., L$
- Easier to learn the score of perturbed versions of the data at each level
- This procedure becomes a continuous-time stochastic process.



Stochastic Differential Equations

- Diffusion processes describe how particles spread over time

Stochastic Differential Equations

- Diffusion processes describe how particles spread over time
- Can be mathematically represented using Stochastic Differential Equations (SDEs):

$$dx = f(x, t)dt + g(t)dw$$

Stochastic Differential Equations

- Diffusion processes describe how particles spread over time
- Can be mathematically represented using Stochastic Differential Equations (SDEs):

$$dx = f(x,t)dt + g(t)dw$$

- Components:
  - $f(.,t) : \mathbb{R}^d \to \mathbb{R}$: Drift coefficient (deterministic part)
  - $g(t) \in \mathbb{R}$: Diffusion coefficient (noise amplitude)
  - dw: Infinitesimal white noise
  - w: Brownian motion
  - $t \in [0, +\infty]$

Forward SDE

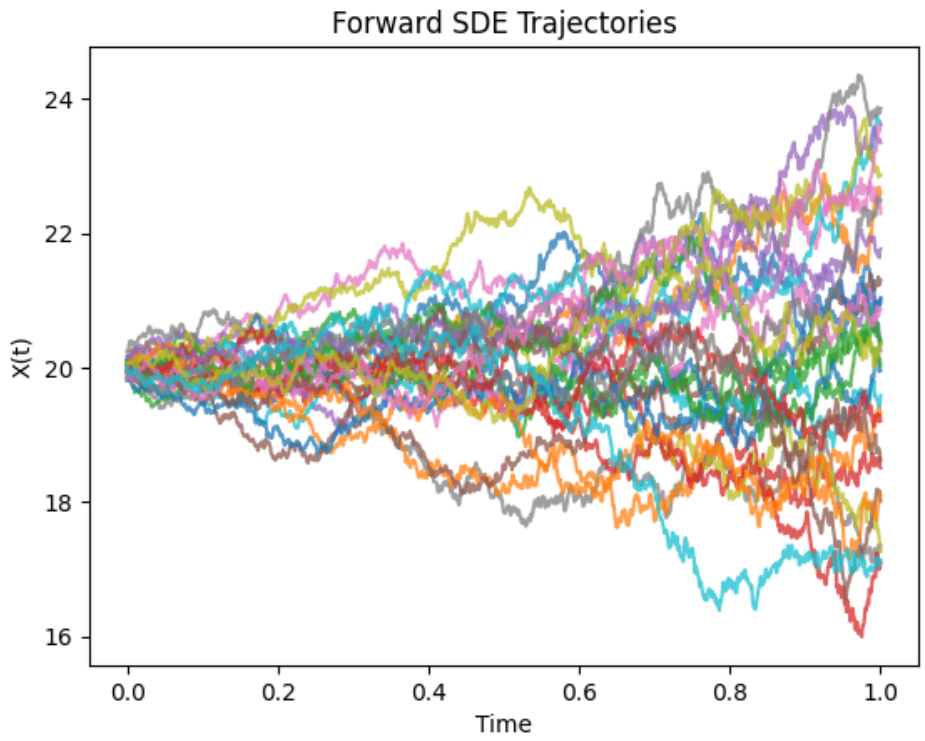


Figure: Forward SDE: $dx = e^t dt$, where $t \in [0, 1]$

- Solution of the SDE is $\{x(T)\}_{t \in [0,T]}$

Forward SDE

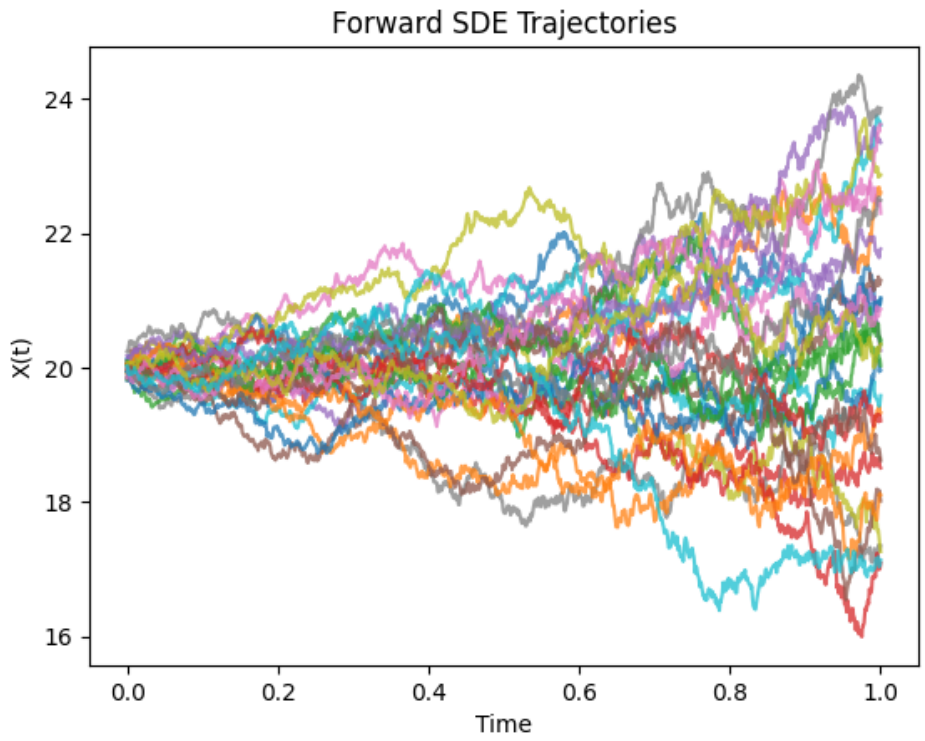


Figure: Forward SDE: $dx = e^t dt$, where $t \in [0, 1]$

- Solution of the SDE is $\{x(T)\}_{t \in [0,T]}$
- Let $p_t(x)$ denote the (marginal) probability density function of $x(t)$.

Forward SDE

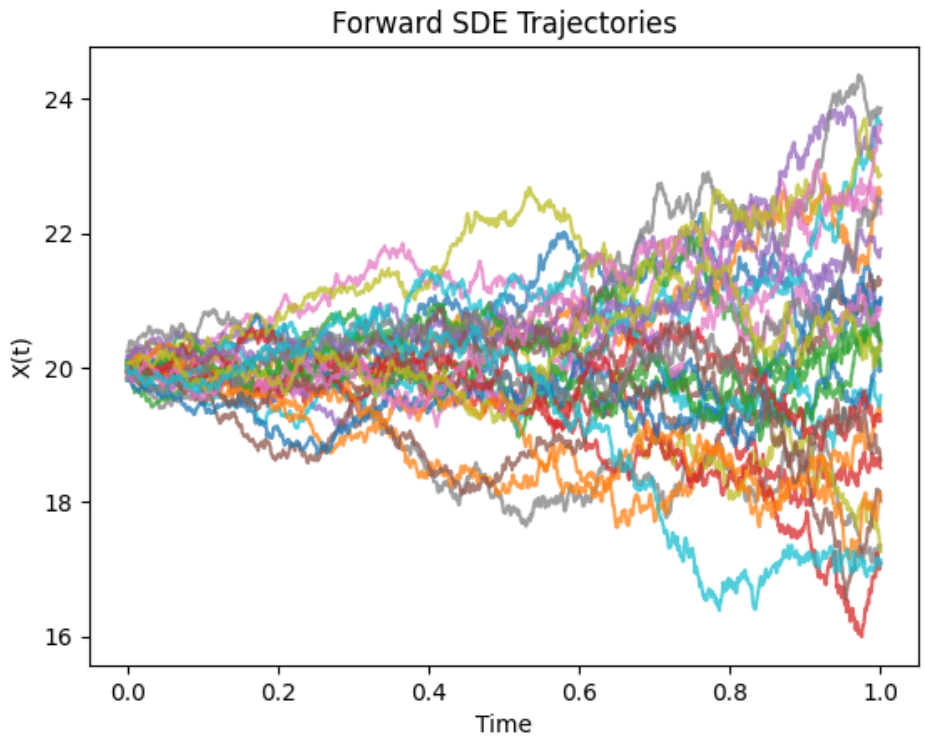


Figure: Forward SDE: $dx = e^t dt$, where $t \in [0, 1]$

- Solution of the SDE is $\{x(T)\}_{t \in [0,T]}$
- Let $p_t(x)$ denote the (marginal) probability density function of $x(t)$.
- $p_0(x)$ is the data distribution and $p_T(x)$ is tractable noise distribution

Recap

1. We start with samples from the data distribution $p_d(\mathbf{x})$.

Recap

1. We start with samples from the data distribution $p_d(\mathbf{x})$.
2. Our goal is to model a good approximation $p_m(\mathbf{x}; \theta)$ that matches $p_d(\mathbf{x})$.

Recap

1. We start with samples from the data distribution $p_d(\mathbf{x})$.
2. Our goal is to model a good approximation $p_m(\mathbf{x}; \theta)$ that matches $p_d(\mathbf{x})$.
3. We approach this by minimizing the Fisher Divergence.

Recap

1. We start with samples from the data distribution $p_d(\mathbf{x})$.
2. Our goal is to model a good approximation $p_m(\mathbf{x}; \theta)$ that matches $p_d(\mathbf{x})$.
3. We approach this by minimizing the Fisher Divergence.
4. To do this, we need to learn the score function $\mathbf{s}_m(\mathbf{x}; \theta) = \nabla_{\mathbf{x}} \log p_m(\mathbf{x}; \theta)$.

Recap

1. We start with samples from the data distribution $p_d(\mathbf{x})$.
2. Our goal is to model a good approximation $p_m(\mathbf{x}; \theta)$ that matches $p_d(\mathbf{x})$.
3. We approach this by minimizing the Fisher Divergence.
4. To do this, we need to learn the score function $\mathbf{s}_m(\mathbf{x}; \theta) = \nabla_{\mathbf{x}} \log p_m(\mathbf{x}; \theta)$.
5. We can learn $\mathbf{s}_m(\mathbf{x}; \theta)$ more effectively through noise perturbation using SDEs.

Recap

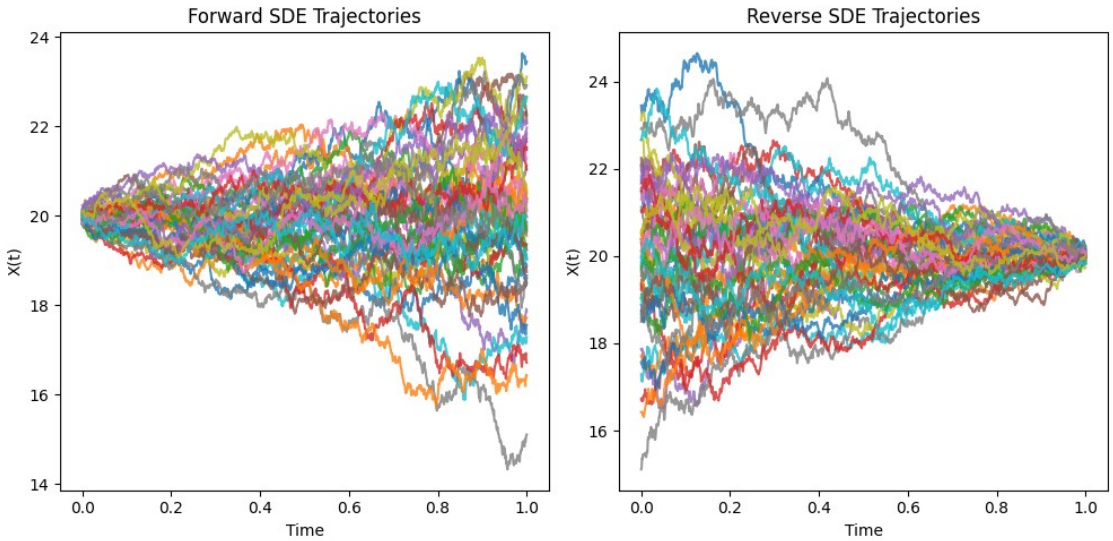
1. We start with samples from the data distribution $p_d(\mathbf{x})$.
2. Our goal is to model a good approximation $p_m(\mathbf{x}; \theta)$ that matches $p_d(\mathbf{x})$.
3. We approach this by minimizing the Fisher Divergence.
4. To do this, we need to learn the score function $\mathbf{s}_m(\mathbf{x}; \theta) = \nabla_{\mathbf{x}} \log p_m(\mathbf{x}; \theta)$.
5. We can learn $\mathbf{s}_m(\mathbf{x}; \theta)$ more effectively through noise perturbation using SDEs.
6. This creates a continuum of distributions from data ($t = 0$) to noise ($t = 1$), namely $p_t(x)$.

Recap

1. We start with samples from the data distribution $p_d(\mathbf{x})$.
2. Our goal is to model a good approximation $p_m(\mathbf{x}; \theta)$ that matches $p_d(\mathbf{x})$.
3. We approach this by minimizing the Fisher Divergence.
4. To do this, we need to learn the score function $\mathbf{s}_m(\mathbf{x}; \theta) = \nabla_{\mathbf{x}} \log p_m(\mathbf{x}; \theta)$.
5. We can learn $\mathbf{s}_m(\mathbf{x}; \theta)$ more effectively through noise perturbation using SDEs.
6. This creates a continuum of distributions from data ($t = 0$) to noise ($t = 1$), namely $p_t(x)$.
7. At each time t, we train a neural network $\mathbf{s}_\theta(\mathbf{x}, t)$ to estimate $\nabla_{\mathbf{x}} \log p_t(\mathbf{x})$.

Reversing the SDE

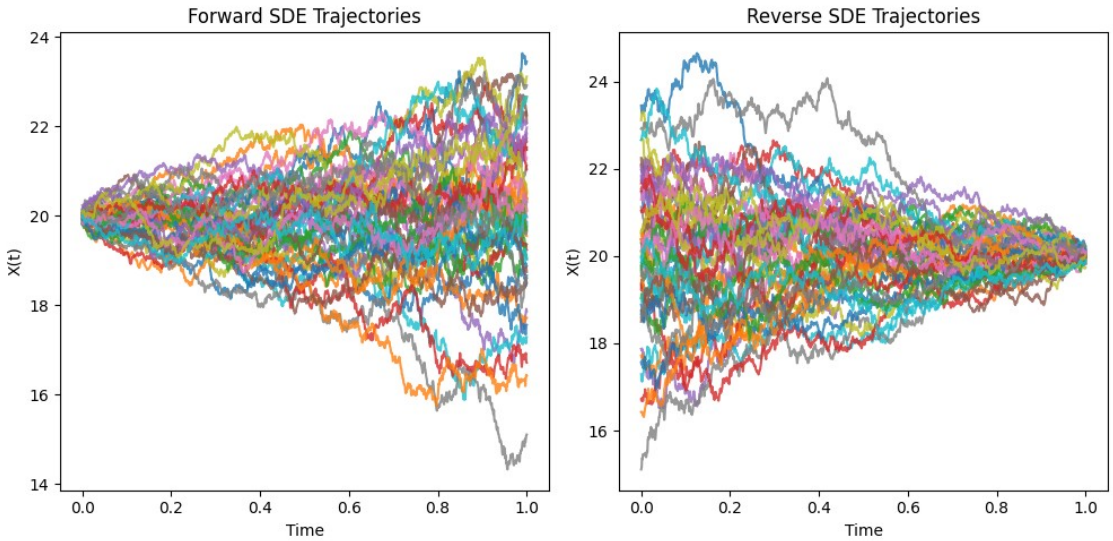
- We can generate samples by reversing the perturbation



Reversing the SDE

- We can generate samples by reversing the perturbation
- Reverse-time SDE (Brian D.O. Anderson, 1979):

$$dx = \left[f(x,t) - g^2(t) \nabla_x \log p_t(x) \right] dt + g(t) d\bar{w}$$

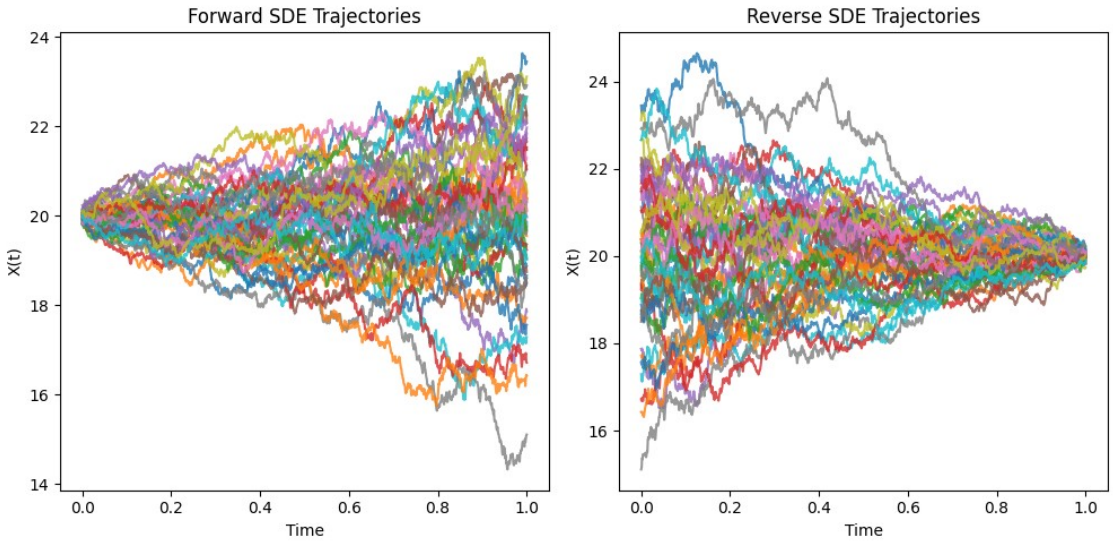


Reversing the SDE

- We can generate samples by reversing the perturbation
- Reverse-time SDE (Brian D.O. Anderson, 1979):

$$dx = \left[f(x,t) - g^2(t) \nabla_x \log p_t(x) \right] dt + g(t) d\bar{w}$$

- $\nabla_x \log p_t(x)$ is called the **score function**

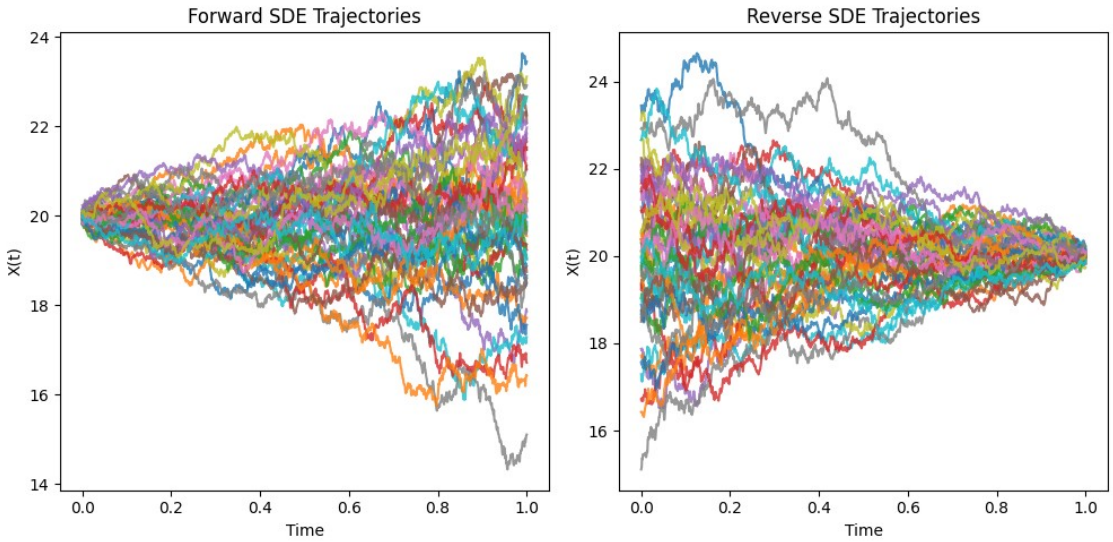


Reversing the SDE

- We can generate samples by reversing the perturbation
- Reverse-time SDE (Brian D.O. Anderson, 1979):

$$dx = \left[f(x,t) - g^2(t) \nabla_x \log p_t(x) \right] dt + g(t) d\bar{w}$$

- $\nabla_x \log p_t(x)$ is called the **score function**
- The score function guides the reverse diffusion process

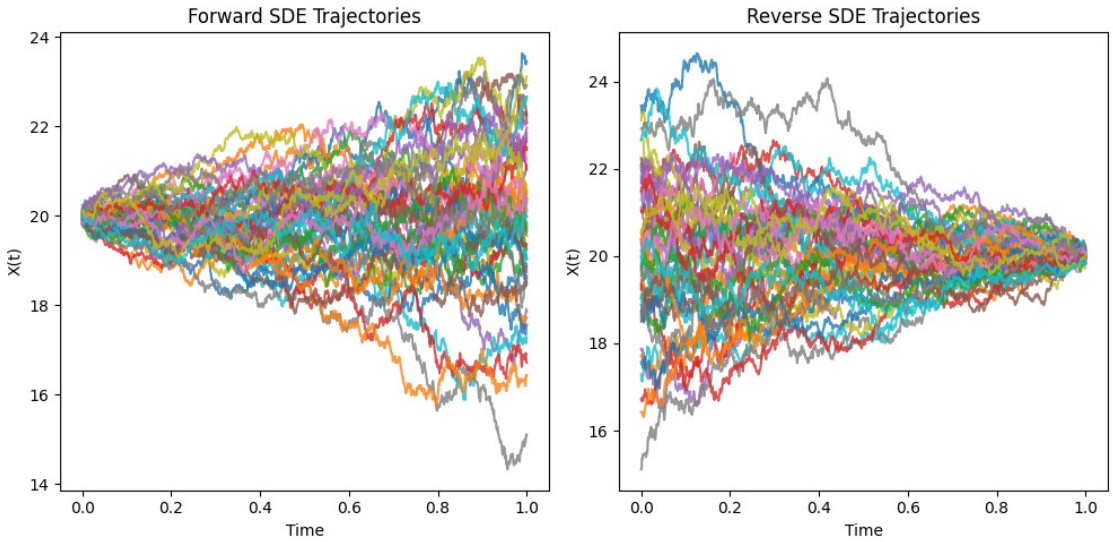


Reversing the SDE

- We can generate samples by reversing the perturbation
- Reverse-time SDE (Brian D.O. Anderson, 1979):

$$dx = \left[f(x,t) - g^2(t) \nabla_x \log p_t(x) \right] dt + g(t) d\bar{w}$$

- $\nabla_x \log p_t(x)$ is called the **score function**
- The score function guides the reverse diffusion process



Target Distribution: Gaussian Mixture Model

- Our target distribution is a two-component GMM:

$$p(x) = w_1 \mathcal{N}(x; \mu_1, \sigma_1^2) + w_2 \mathcal{N}(x; \mu_2, \sigma_2^2)$$

Target Distribution: Gaussian Mixture Model

- Our target distribution is a two-component GMM:

$$p(x) = w_1 \mathcal{N}(x; \mu_1, \sigma_1^2) + w_2 \mathcal{N}(x; \mu_2, \sigma_2^2)$$

- Parameters:

$$\mu_1 = 1.0, \quad \sigma_1 = 0.5, \quad w_1 = 0.2$$
$$\mu_2 = 6.0, \quad \sigma_2 = 1.0, \quad w_2 = 0.8$$

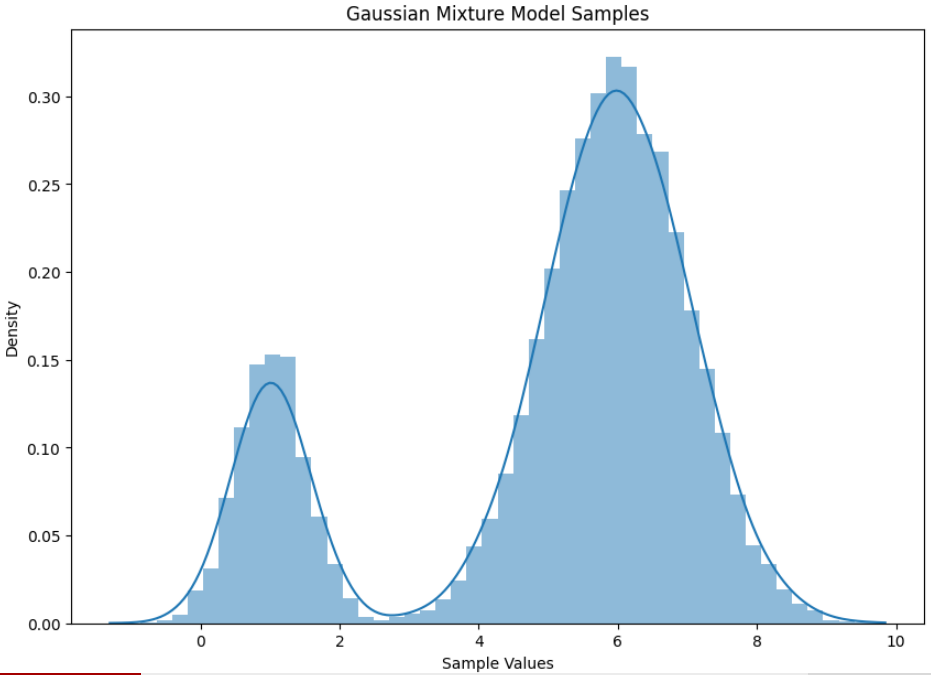
Target Distribution: Gaussian Mixture Model

- Our target distribution is a two-component GMM:

$$p(x) = w_1 \mathcal{N}(x; \mu_1, \sigma_1^2) + w_2 \mathcal{N}(x; \mu_2, \sigma_2^2)$$

- Parameters:

$$\mu_1 = 1.0, \quad \sigma_1 = 0.5, \quad w_1 = 0.2$$
$$\mu_2 = 6.0, \quad \sigma_2 = 1.0, \quad w_2 = 0.8$$



Forward Diffusion Process

- $x(0) \sim p_0$, we apply diffusion:

$$dx = g(t)dw$$
$$\text{where } g(t) = e^t$$

Forward Diffusion Process

- $x(0) \sim p_0$, we apply diffusion:

$$dx = g(t)dw$$
$$\text{where } g(t) = e^t$$

- This SDE perturbs data over time, with solution:

$$x(t) = x(0) + dx$$

Forward Diffusion Process

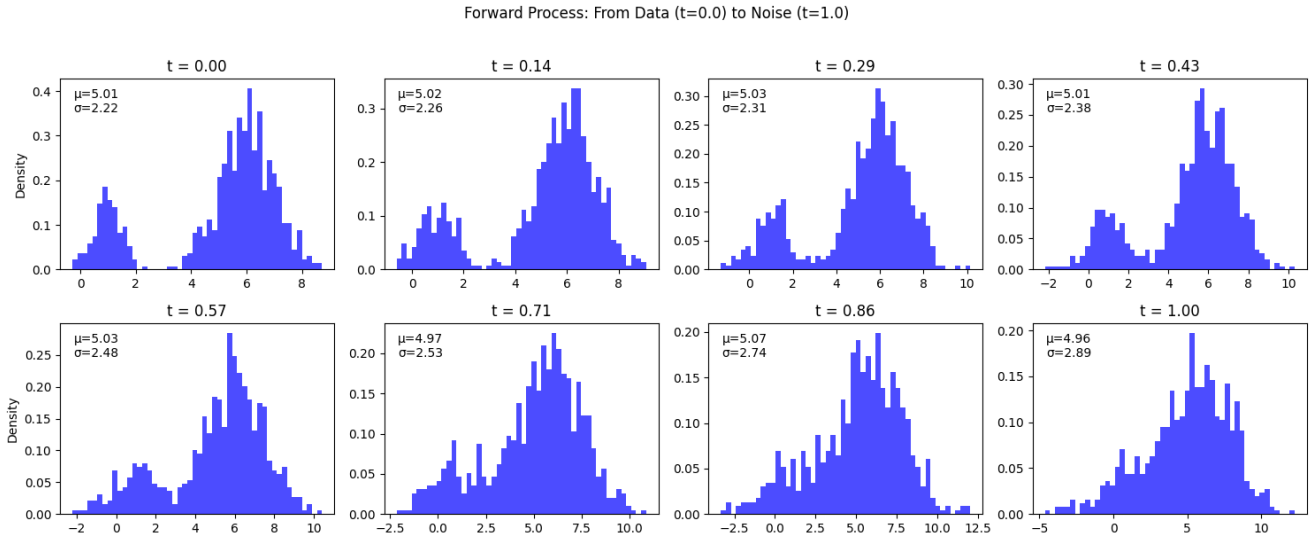
- $x(0) \sim p_0$, we apply diffusion:

$$dx = g(t)dw$$

$$\text{where } g(t) = e^t$$

- This SDE perturbs data over time, with solution:

$$x(t) = x(0) + dx$$



Learning the Score Function

- We train a neural network $s_\theta(x, t)$ to estimate the score function

Learning the Score Function

- We train a neural network $s_\theta(x, t)$ to estimate the score function
- Minimize Fisher divergence (score matching loss)

Learning the Score Function

- We train a neural network $s_\theta(x, t)$ to estimate the score function
- Minimize Fisher divergence (score matching loss)
- For our GMM, we can compute the true score at any time t:

$$\nabla_x \log p_t(x) = \frac{w_1 p_{1,t}(x) \cdot \frac{-(x-\mu_1)}{\sigma_{1,t}^2} + w_2 p_{2,t}(x) \cdot \frac{-(x-\mu_2)}{\sigma_{2,t}^2}}{w_1 p_{1,t}(x) + w_2 p_{2,t}(x)}$$

Learning the Score Function

- We train a neural network $s_\theta(x, t)$ to estimate the score function
- Minimize Fisher divergence (score matching loss)
- For our GMM, we can compute the true score at any time t:

$$\nabla_x \log p_t(x) = \frac{w_1 p_{1,t}(x) \cdot \frac{-(x-\mu_1)}{\sigma_{1,t}^2} + w_2 p_{2,t}(x) \cdot \frac{-(x-\mu_2)}{\sigma_{2,t}^2}}{w_1 p_{1,t}(x) + w_2 p_{2,t}(x)}$$

- True score allows us to train our network

Reversing the Diffusion Process

- To reverse the diffusion, we need the time-dependent score function

Reversing the Diffusion Process

- To reverse the diffusion, we need the time-dependent score function
- Reverse SDE:

$$dx = \left[f(x,t) - g^2(t) \nabla_x \log p_t(x) \right] dt + g(t) d\bar{w}$$

Reversing the Diffusion Process

- To reverse the diffusion, we need the time-dependent score function
- Reverse SDE:

$$dx = \left[f(x,t) - g^2(t) \nabla_x \log p_t(x) \right] dt + g(t) d\bar{w}$$

- We replace the true score with our learned score estimator $s_\theta(x,t)$

$$dx = \left[f(x,t) - g^2(t) s_\theta(x,t) \right] dt + g(t) d\bar{w}$$

Reversing the Diffusion Process

- To reverse the diffusion, we need the time-dependent score function
- Reverse SDE:

$$dx = \left[f(x,t) - g^2(t) \nabla_x \log p_t(x) \right] dt + g(t) d\bar{w}$$

- We replace the true score with our learned score estimator $s_\theta(x,t)$

$$dx = \left[f(x,t) - g^2(t) s_\theta(x,t) \right] dt + g(t) d\bar{w}$$

- Where our neural network was trained to minimize:

$$L(\theta) = \mathbb{E}\left[\| s_\theta(x(t),t) - \nabla_x \log p_t(x(t)) \|^2 \right]$$

Score Function Estimation

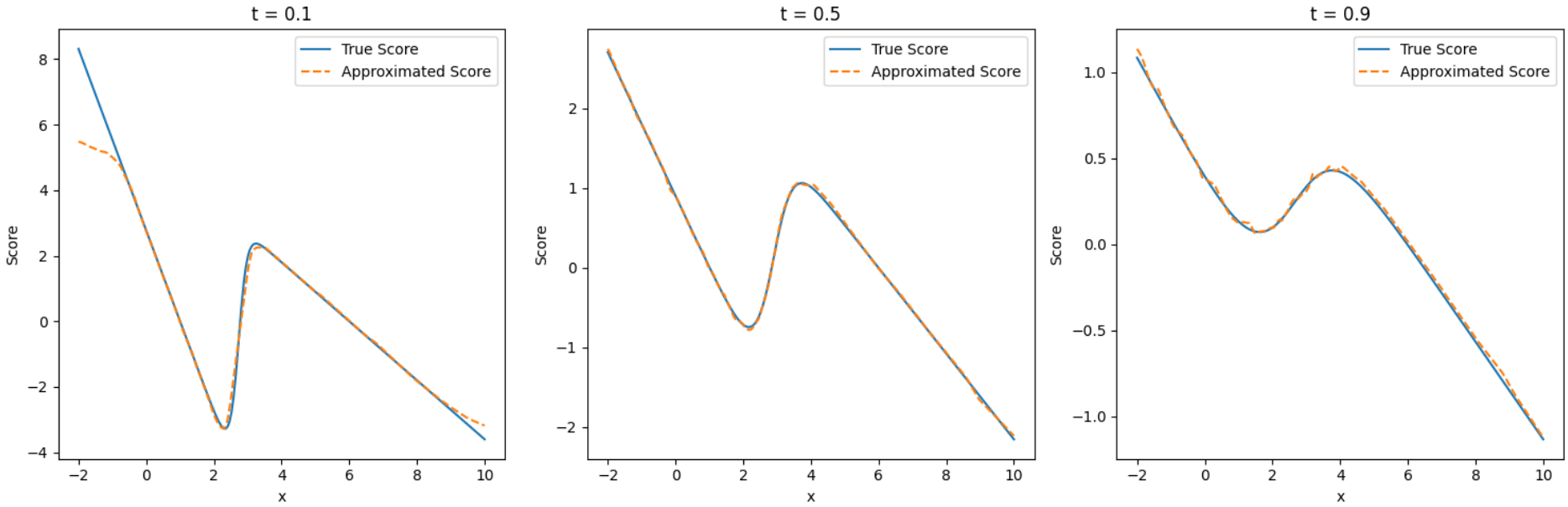


Figure: Comparison of true score vs. learned score at different time steps

Data Recovery Results

- Using the learned score function and Euler-Maruyama method to generate samples via:

$$x_{t-\Delta t} = x_t - f(x_t, t)\Delta t + g^2(t)s_\theta(x_t, t)\Delta t - g(t)\sqrt{\Delta t}z_t$$

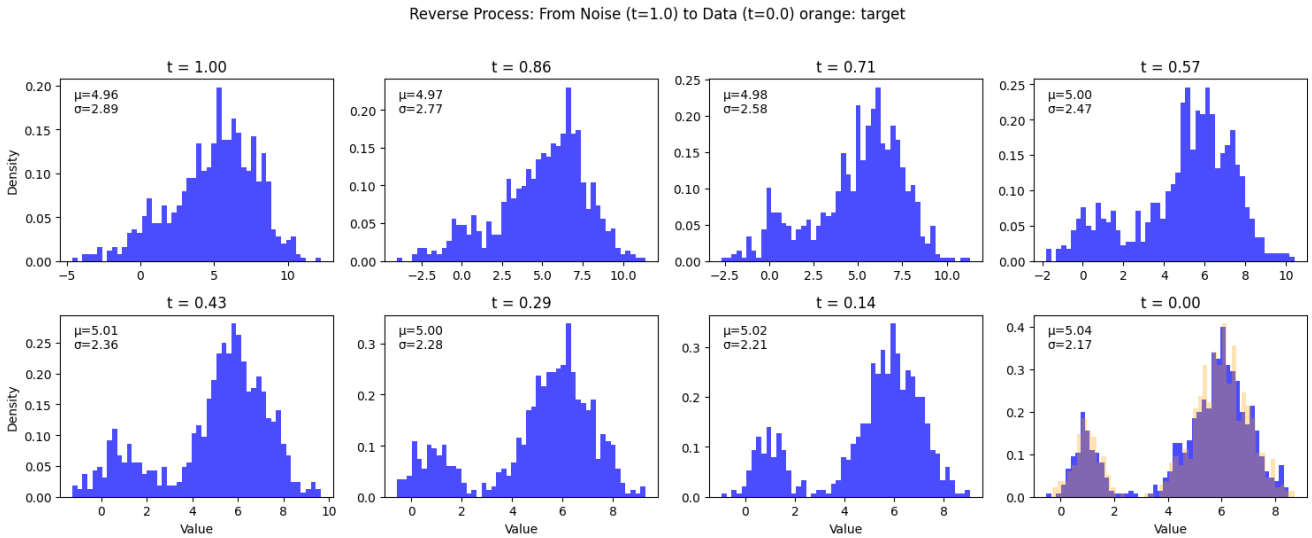
where $z_t \sim \mathcal{N}(0, 1)$ and sampling from t $= 1$ to t $= 0$

Data Recovery Results

- Using the learned score function and Euler-Maruyama method to generate samples via:

$$x_{t-\Delta t} = x_t - f(x_t, t)\Delta t + g^2(t)s_\theta(x_t, t)\Delta t - g(t)\sqrt{\Delta t}z_t$$

where $z_t \sim \mathcal{N}(0, 1)$ and sampling from t $= 1$ to t $= 0$



Practical Implementation of Score Matching

- **Challenge**: No access to $\mathbf{s}_d(\mathbf{x})$ directly, only samples from p_d

Practical Implementation of Score Matching

- **Challenge**: No access to $\mathbf{s}_d(\mathbf{x})$ directly, only samples from p_d
- **Solution**: Hyvärinen (2005) showed that:

$$L(\theta) = J(\theta) + C$$

$$J(\theta) = \mathbb{E}_{p_d}\left[\text{tr}(\nabla_{\mathbf{x}}\mathbf{s}_m(\mathbf{x};\theta)) + \frac{1}{2}\|\mathbf{s}_m(\mathbf{x};\theta)\|_2^2\right]$$

where C is a constant independent of θ and $\nabla_{\mathbf{x}}\mathbf{s}_m$ is the Hessian

Practical Implementation of Score Matching

- **Challenge**: No access to $\mathbf{s}_d(\mathbf{x})$ directly, only samples from p_d
- **Solution**: Hyvärinen (2005) showed that:

$$L(\theta) = J(\theta) + C$$

$$J(\theta) = \mathbb{E}_{p_d}\left[\text{tr}(\nabla_{\mathbf{x}}\mathbf{s}_m(\mathbf{x};\theta)) + \frac{1}{2}\|\mathbf{s}_m(\mathbf{x};\theta)\|_2^2\right]$$

where C is a constant independent of θ and $\nabla_{\mathbf{x}}\mathbf{s}_m$ is the Hessian

- This gives us an unbiased estimator:

$$\hat{J}(\theta;\mathbf{x}_1^N) = \frac{1}{N}\sum_{i=1}^{N}\left[\text{tr}(\nabla_{\mathbf{x}}\mathbf{s}_m(\mathbf{x}_i;\theta)) + \frac{1}{2}\|\mathbf{s}_m(\mathbf{x}_i;\theta)\|_2^2\right]$$

Practical Implementation of Score Matching

- **Challenge**: No access to $\mathbf{s}_d(\mathbf{x})$ directly, only samples from p_d
- **Solution**: Hyvärinen (2005) showed that:

$$L(\theta) = J(\theta) + C$$

$$J(\theta) = \mathbb{E}_{p_d}\left[\text{tr}(\nabla_{\mathbf{x}}\mathbf{s}_m(\mathbf{x};\theta)) + \frac{1}{2}\|\mathbf{s}_m(\mathbf{x};\theta)\|_2^2\right]$$

  where C is a constant independent of θ and $\nabla_{\mathbf{x}}\mathbf{s}_m$ is the Hessian
- This gives us an unbiased estimator:

$$\hat{J}(\theta;\mathbf{x}_1^N) = \frac{1}{N}\sum_{i=1}^{N}\left[\text{tr}(\nabla_{\mathbf{x}}\mathbf{s}_m(\mathbf{x}_i;\theta)) + \frac{1}{2}\|\mathbf{s}_m(\mathbf{x}_i;\theta)\|_2^2\right]$$

- Computational challenge: Computing the trace of Hessian requires $O(D)$ backward passes

Pizza time!

- Project high-dimensional scores onto random directions to reduce complexity, i.e, take slices

Pizza time!

- Project high-dimensional scores onto random directions to reduce complexity, i.e, take slices
- Sliced Fisher divergence (Song et.al, 2019):

$$L(\theta; p_v) = \frac{1}{2}\mathbb{E}_{p_v}\mathbb{E}_{p_d}\left[(v^\top \mathbf{s}_m(\mathbf{x};\theta) - v^\top \mathbf{s}_d(\mathbf{x}))^2\right]$$

where $v \sim p_v$ (e.g., standard normal)

Pizza time!

- Project high-dimensional scores onto random directions to reduce complexity, i.e, take slices
- Sliced Fisher divergence (Song et.al, 2019):

$$L(\theta; p_v) = \frac{1}{2}\mathbb{E}_{p_v}\mathbb{E}_{p_d}\left[(v^\top \mathbf{s}_m(\mathbf{x};\theta) - v^\top \mathbf{s}_d(\mathbf{x}))^2\right]$$

  where $v \sim p_v$ (e.g., standard normal)
- By applying this to Hyvärinen's work, we don't need true scores $\mathbf{s}_d(\mathbf{x})$:

$$J(\theta; p_v) = \mathbb{E}_{p_v}\mathbb{E}_{p_d}\left[v^\top \nabla_{\mathbf{x}} \mathbf{s}_m(\mathbf{x};\theta) v + \frac{1}{2}(v^\top \mathbf{s}_m(\mathbf{x};\theta))^2\right]$$

Pizza time!

- Project high-dimensional scores onto random directions to reduce complexity, i.e, take slices
- Sliced Fisher divergence (Song et.al, 2019):

$$L(\theta; p_v) = \frac{1}{2}\mathbb{E}_{p_v}\mathbb{E}_{p_d}\left[(v^\top \mathbf{s}_m(\mathbf{x};\theta) - v^\top \mathbf{s}_d(\mathbf{x}))^2\right]$$

  where $v \sim p_v$ (e.g., standard normal)
- By applying this to Hyvärinen's work, we don't need true scores $\mathbf{s}_d(\mathbf{x})$:

$$J(\theta; p_v) = \mathbb{E}_{p_v}\mathbb{E}_{p_d}\left[v^\top \nabla_{\mathbf{x}} \mathbf{s}_m(\mathbf{x};\theta) v + \frac{1}{2}(v^\top \mathbf{s}_m(\mathbf{x};\theta))^2\right]$$

- This only requires Hessian-vector products - computationally efficient!

Let's try this again

- Going back to our Gaussian mixture distribution

Let's try this again

- Going back to our Gaussian mixture distribution

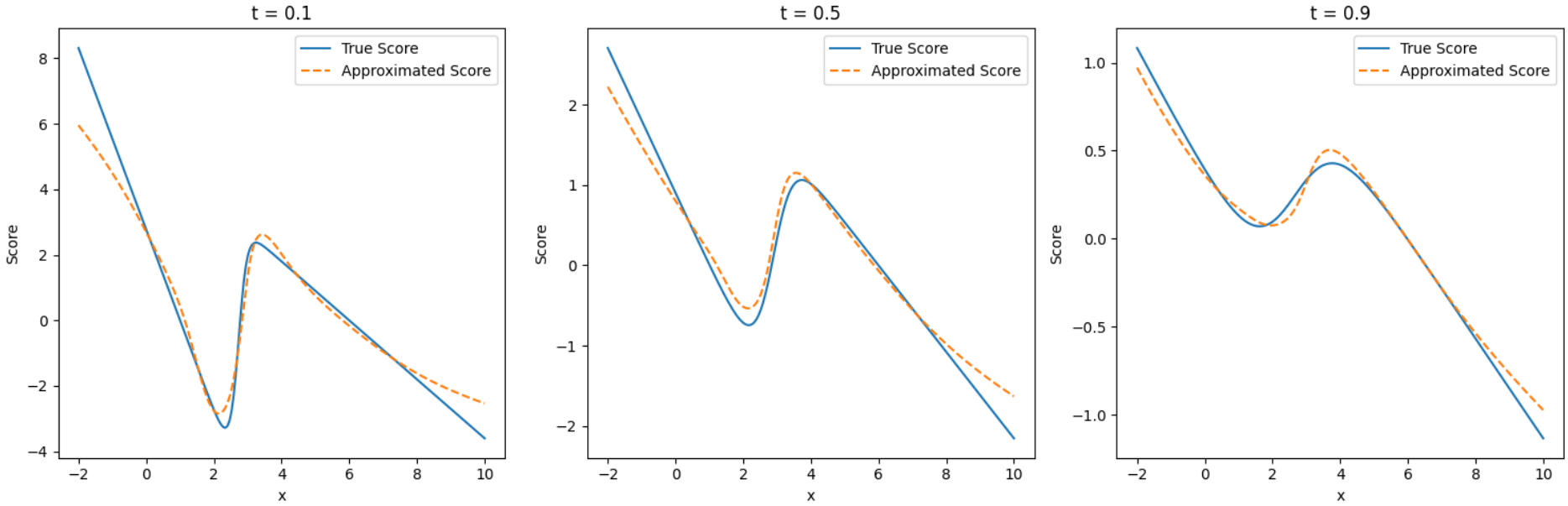
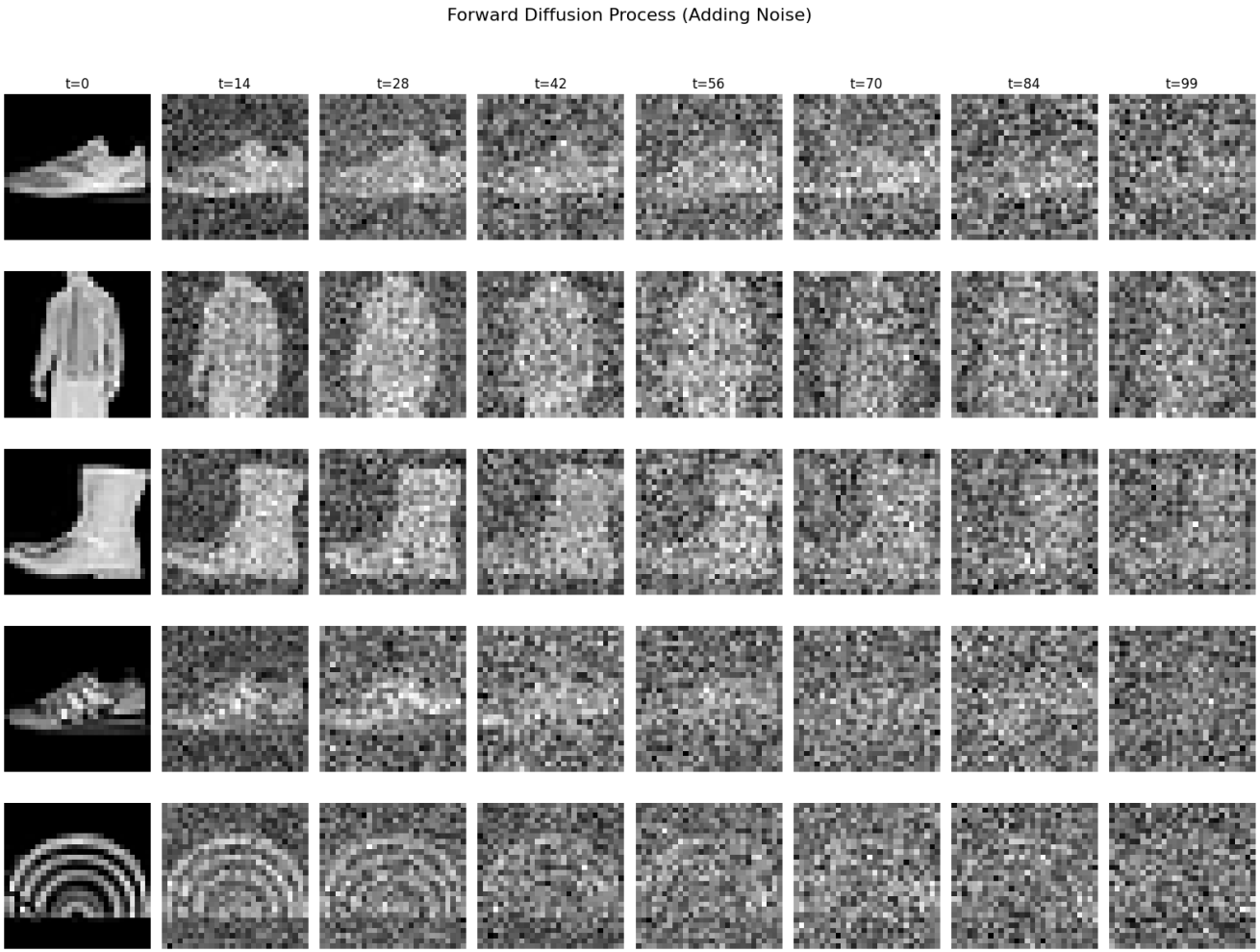
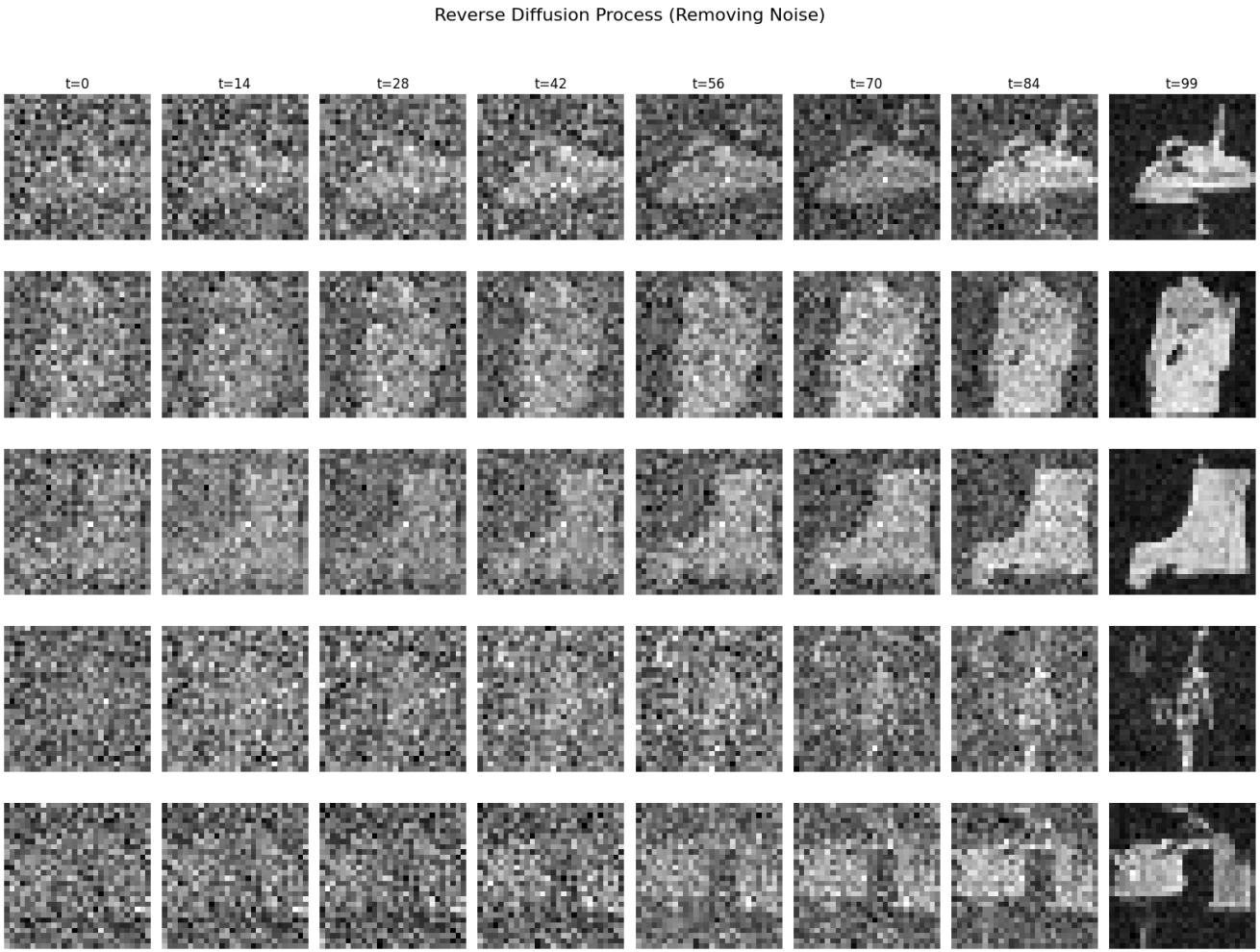


Figure: Comparison of true scores and approximate scores via sliced score matching

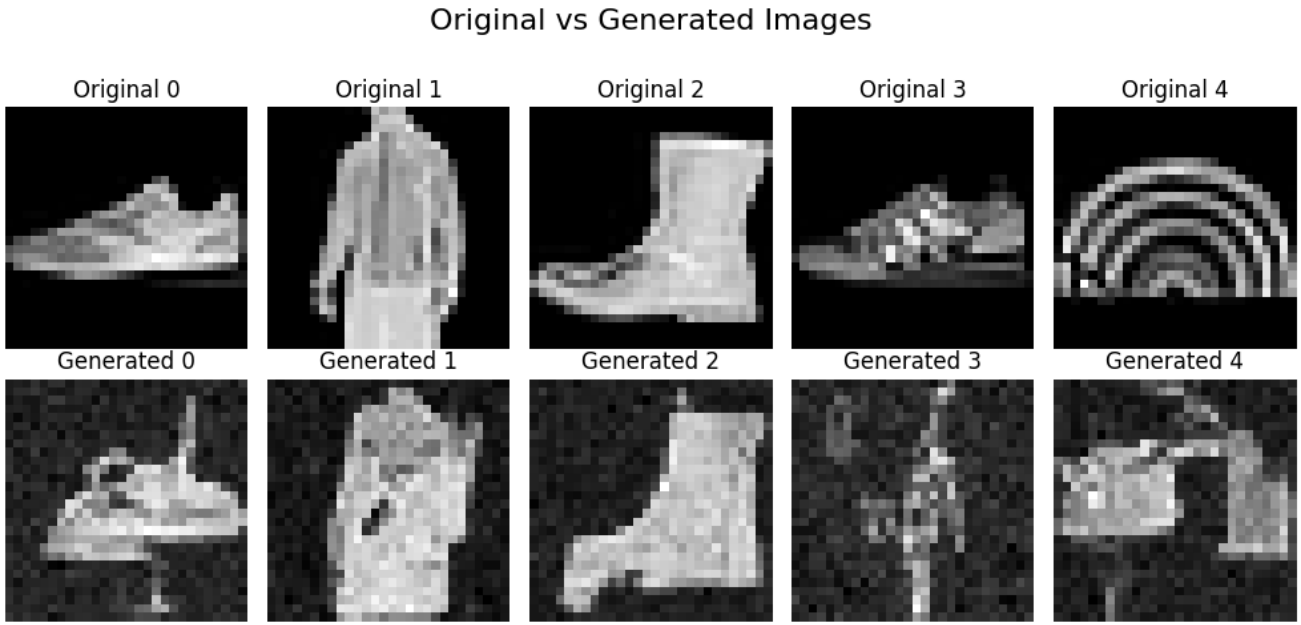
Higher dimensions?



Higher dimensions?



Higher dimensions?



Supercharged Score-Matched Image

Figure: Generated images using U-Net architecture

Are we there yet?

THE END.

References

Anderson, B. D. (1982). Reverse-time diffusion equation models. *Stochastic Processes and their Applications*, 12(3):313–326.

Chwialkowski, K., Strathmann, H., and Gretton, A. (2016). A kernel test of goodness of fit. *Proceedings of The 33rd International Conference on Machine Learning*, 48:2606–2615.

Dinh, L., Sohl-Dickstein, J., and Bengio, S. (2017). Density estimation using real nvp. *International Conference on Learning Representations*.

Hyvarinen, A. (2005). Estimation of non-normalized statistical models by score matching. *Journal of Machine Learning Research*, 6:695–709.

Kingma, D. P. and Welling, M. (2014). Auto-encoding variational bayes. *International Conference on Learning Representations*.

Larochelle, H. and Murray, I. (2011). The neural autoregressive distribution estimator. *International Conference on Artificial Intelligence and Statistics*, pages 29–37.

Metz, L., Poole, B., Pfau, D., and Sohl-Dickstein, J. (2017). Unrolled generative adversarial networks. *5th International Conference on Learning Representations, ICLR 2017, Toulon, France, April 24-26,*

Questions?

More generated images

(a) ODE Sampler

(b) Euler-Maruyama Sampler

(a) Predictive-Corrector Sampler

Practical Implementation of Sliced Score Matching

dataset $\mathbf{x}_1^N$, we draw M random projection vectors $\mathbf{v}_{ij}$ for each $\mathbf{x}_i$

Algorithm 0: Sliced Score Matching

- **Input:** $\tilde{p}_m(\cdot;\theta)$, $\mathbf{x}$, $\mathbf{v}$

  $\mathbf{s}_m(\mathbf{x};\theta) \leftarrow \text{grad}(\log \tilde{p}_m(\mathbf{x};\theta), \mathbf{x})$

  $\mathbf{v}^\top \nabla_{\mathbf{x}} \mathbf{s}_m(\mathbf{x};\theta)\mathbf{v} \leftarrow \text{grad}(\mathbf{v}^\top \mathbf{s}_m(\mathbf{x};\theta), \mathbf{x})$

  $J \leftarrow \frac{1}{2}(\mathbf{v}^\top \mathbf{s}_m(\mathbf{x};\theta))^2$

  $J \leftarrow J + \mathbf{v}^\top \nabla_{\mathbf{x}} \mathbf{s}_m(\mathbf{x};\theta)\mathbf{v}$

  return J

Denoising Score Matching

- Another approach to bypass the Hessian computation in score matching

Denoising Score Matching

- Another approach to bypass the Hessian computation in score matching
- For perturbed data: $\tilde{\mathbf{x}} = \mathbf{x} + \sigma\epsilon$ where $\epsilon \sim \mathcal{N}(0, I)$

Denoising Score Matching

- Another approach to bypass the Hessian computation in score matching
- For perturbed data: $\tilde{\mathbf{x}} = \mathbf{x} + \sigma\epsilon$ where $\epsilon \sim \mathcal{N}(0, I)$
- Vincent (2011) showed that:

$$\nabla_{\tilde{\mathbf{x}}} \log p_\sigma(\tilde{\mathbf{x}}) \approx \frac{\mathbf{x} - \tilde{\mathbf{x}}}{\sigma^2}$$

Denoising Score Matching

- Another approach to bypass the Hessian computation in score matching
- For perturbed data: $\tilde{\mathbf{x}} = \mathbf{x} + \sigma\epsilon$ where $\epsilon \sim \mathcal{N}(0, I)$
- Vincent (2011) showed that:

$$\nabla_{\tilde{\mathbf{x}}} \log p_\sigma(\tilde{\mathbf{x}}) \approx \frac{\mathbf{x} - \tilde{\mathbf{x}}}{\sigma^2}$$

- Denoising score matching objective:

$$J_{\text{DSM}}(\theta) = \mathbb{E}_{p_d(\mathbf{x})} \mathbb{E}_{p_\sigma(\tilde{\mathbf{x}}|\mathbf{x})} \left[\left\| \mathbf{s}_\theta(\tilde{\mathbf{x}}) - \frac{\mathbf{x} - \tilde{\mathbf{x}}}{\sigma^2} \right\|^2 \right]$$

Denoising Score Matching

- Another approach to bypass the Hessian computation in score matching
- For perturbed data: $\tilde{\mathbf{x}} = \mathbf{x} + \sigma\epsilon$ where $\epsilon \sim \mathcal{N}(0, I)$
- Vincent (2011) showed that:

$$\nabla_{\tilde{\mathbf{x}}} \log p_\sigma(\tilde{\mathbf{x}}) \approx \frac{\mathbf{x} - \tilde{\mathbf{x}}}{\sigma^2}$$

- Denoising score matching objective:

$$J_{\text{DSM}}(\theta) = \mathbb{E}_{p_d(\mathbf{x})} \mathbb{E}_{p_\sigma(\tilde{\mathbf{x}}|\mathbf{x})} \left[\left\| \mathbf{s}_\theta(\tilde{\mathbf{x}}) - \frac{\mathbf{x} - \tilde{\mathbf{x}}}{\sigma^2} \right\|^2 \right]$$

- No second derivatives required $\implies$ computationally efficient

Denoising Score Matching

- Another approach to bypass the Hessian computation in score matching
- For perturbed data: $\tilde{\mathbf{x}} = \mathbf{x} + \sigma\epsilon$ where $\epsilon \sim \mathcal{N}(0, I)$
- Vincent (2011) showed that:

$$\nabla_{\tilde{\mathbf{x}}} \log p_\sigma(\tilde{\mathbf{x}}) \approx \frac{\mathbf{x} - \tilde{\mathbf{x}}}{\sigma^2}$$

- Denoising score matching objective:

$$J_{\text{DSM}}(\theta) = \mathbb{E}_{p_d(\mathbf{x})} \mathbb{E}_{p_\sigma(\tilde{\mathbf{x}}|\mathbf{x})} \left[\left\| \mathbf{s}_\theta(\tilde{\mathbf{x}}) - \frac{\mathbf{x} - \tilde{\mathbf{x}}}{\sigma^2} \right\|^2 \right]$$

- No second derivatives required $\implies$ computationally efficient
- Hard to select optimal noise level without prior knowledge

this again?

- Gaussian mixture distribution scores using DSM

¡2-¿

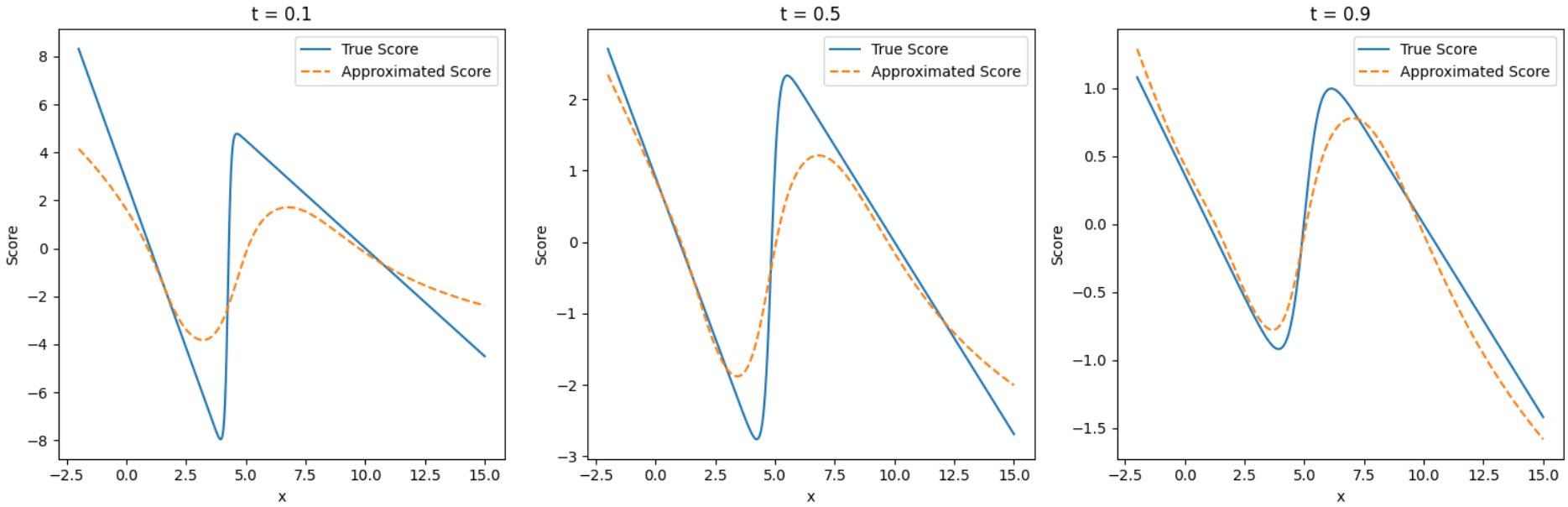


Figure: Comparison of True scores and approximate scores via denoised score matching